AI/ML-Based Predictive Maintenance and Vehicle Health Monitoring System

Module 3 (Connectivity) Project

Submitted by	Joyeeta Das
Enrollment Number	12023002001082
Programme	B.Tech in Computer Science and Engineering (CSE)
Institute	Institute of Engineering and Management, Kolkata

Project 1 selected from the Module 3 assignment

1. Project Overview

This project implements an AI/ML-based predictive maintenance and vehicle health monitoring pipeline. It uses nine vehicle sensor inputs to estimate the probability of component/vehicle failure before it occurs. The predicted probability is converted into a Vehicle Health Score and a maintenance status so that a fleet operator can take preventive action.

The submitted implementation compares Logistic Regression, Decision Tree, Random Forest and XGBoost when available. If XGBoost is unavailable, Gradient Boosting is used as a fallback. An optional LSTM script is also included for sequence-based modelling. Explainable AI is implemented at both global and per-vehicle levels.

2. Objective

To develop a machine-learning system that predicts vehicle/component failure before it occurs using sensor telemetry and translates the prediction into an interpretable health score, maintenance status and major contributing factors.

3. AI/ML Technologies and Models

Technologies required by the assignment and represented in the implementation:

- Random Forest - ensemble tree classifier used in the model comparison.
- XGBoost - used automatically when the xgboost package is installed; otherwise Gradient Boosting is used as a substitute.
- LSTM - optional TensorFlow sequence model using windows of 10 sensor readings.
- Explainable AI (XAI) - global permutation importance and local occlusion-based attribution.

Additional baseline/comparison models implemented: Logistic Regression and Decision Tree. GRU and Transformer are listed among the assignment's suggested model choices, but they are not implemented in the submitted code and therefore are not claimed as project results.

4. Input Data

The system uses the following nine sensor channels:

- Engine temperature
- RPM
- Oil pressure
- Vibration
- Battery voltage
- Coolant temperature
- Fuel consumption
- Vehicle speed
- Operating hours

5. System Architecture

Vehicle Sensors
↓
RPM | Temperature | Pressure | Vibration | Battery | Coolant | Fuel | Speed | Operating Hours
↓
Data Preprocessing (standardization)
↓
ML / DL Model
↓
Failure Probability
↓
Vehicle Health Score
↓
Maintenance Alert + XAI Contributing Factors

6. Methodology

6.1 Dataset Generation

No real fleet telemetry was supplied with the assignment. The code therefore generates a synthetic dataset of 8,000 vehicle records. Approximately 30% are assigned a developing-fault condition with varying severity. A wear factor based on operating hours is also included. The generated data is saved as outputs/vehicle_sensor_data.csv.

6.2 Preprocessing

The dataset is split into training and test sets using an 80:20 stratified split with random_state=42. Sensor features are standardized using StandardScaler fitted on the training data.

6.3 Model Training and Selection

Logistic Regression, Decision Tree and Random Forest are trained and evaluated. The training script attempts to import XGBoost; when it is not installed, Gradient Boosting is used as a fallback. The model with the highest ROC-AUC is saved automatically in models/best_model.joblib.

6.4 Optional LSTM

The included lstm_model.py builds pseudo-sequences of 10 readings, trains a 64-unit LSTM with dropout, and outputs a binary failure probability. TensorFlow is required to run this optional model.

6.5 Vehicle Health Score and Alert Logic

Vehicle Health (%) = 100 × (1 − failure probability). The implementation assigns HEALTHY when failure probability is below 40%, WARNING from 40% to below 75%, and CRITICAL at 75% or above.

6.6 Explainable AI

Global explainability uses permutation feature importance scored by ROC-AUC. Local explainability uses an occlusion approach: one sensor at a time is reset to a predefined healthy reference value, and the reduction in predicted failure probability is measured. The top positive reductions are reported as the vehicle's major contributing factors.

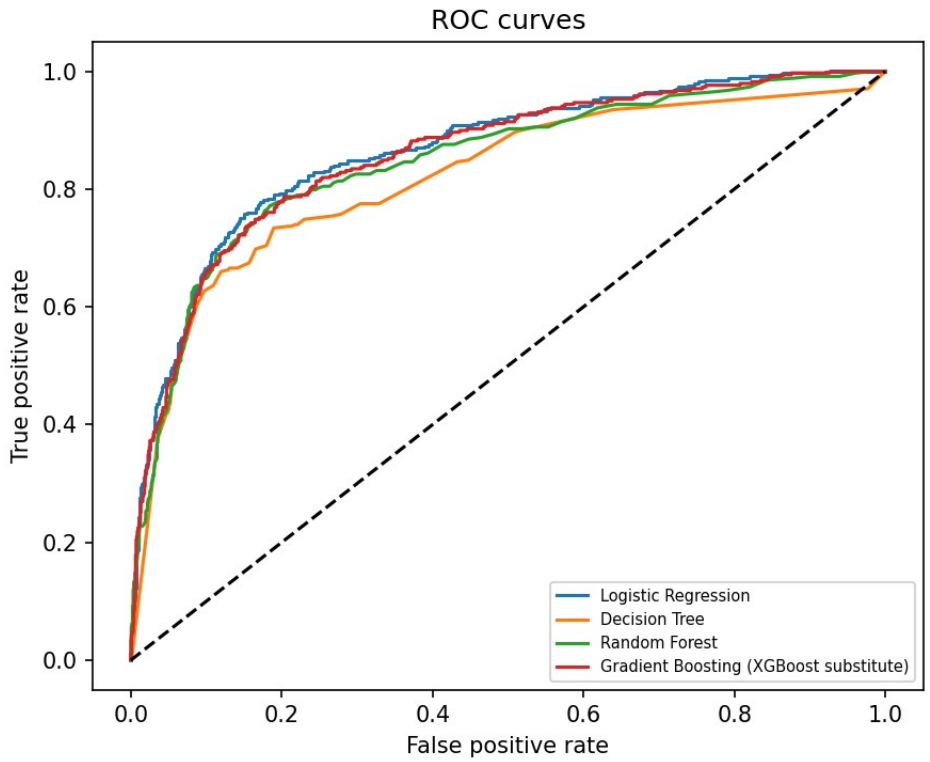
7. Model Evaluation Results

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.823	0.56	0.76	0.645	0.865
Gradient Boosting (XGBoost substitute)	0.849	0.656	0.598	0.625	0.858
Random Forest	0.847	0.669	0.544	0.6	0.846
Decision Tree	0.794	0.509	0.734	0.601	0.819

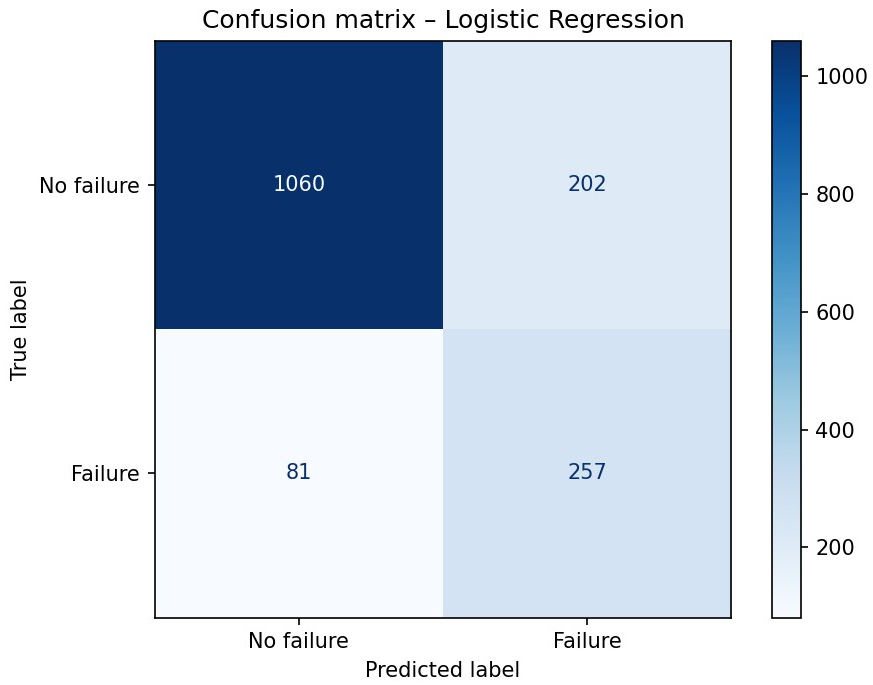
In this recorded run, Logistic Regression achieved the highest ROC-AUC (0.865) and was therefore saved as the deployed model. Gradient Boosting was used as the XGBoost substitute in this run.

8. Evaluation Charts

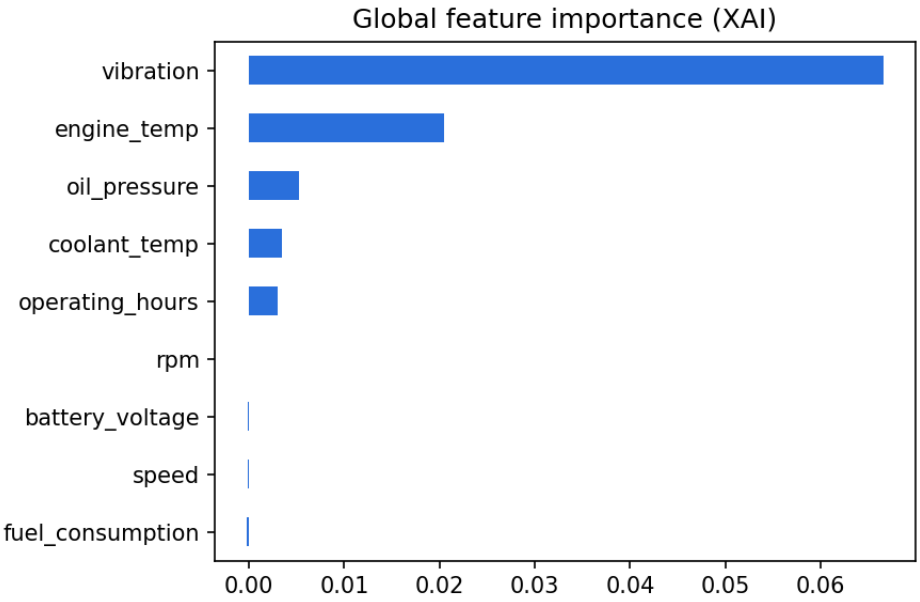
8.1 ROC Curves



8.2 Confusion Matrix - Selected Model



8.3 Global Feature Importance (XAI)



The recorded permutation-importance chart shows vibration as the strongest global feature for the selected model, followed by engine temperature, with smaller contributions from oil pressure, coolant temperature and operating hours.

9. Sample Output

Vehicle Health	46.2%
Failure Probability	53.8%
Status	WARNING

Major contributing factors:

1. High vibration (+29% contribution to predicted risk)
2. Rising engine temperature (+14% contribution to predicted risk)
3. Abnormal oil pressure (+8% contribution to predicted risk)

This sample is generated by the code for a stressed vehicle input. The health score and failure probability are complementary by design, so 46.2% health corresponds to 53.8% failure probability.

10. Streamlit Dashboard

The app.py file provides an interactive Streamlit dashboard. Users can adjust all nine sensor values with sliders. The dashboard displays Vehicle Health, Failure Probability, status (HEALTHY/WARNING/CRITICAL), and the major contributing factors returned by the XAI routine.

11. Limitations

- The reported metrics are based on synthetic data, not real fleet telemetry, and must be re-validated on real-world data before production use.
- The recorded training run used Gradient Boosting as the XGBoost substitute; results should not be presented as XGBoost results.
- The LSTM implementation is optional and requires TensorFlow; its evaluation metrics are not included in the recorded model-comparison CSV.
- GRU and Transformer models are suggested by the assignment but are not implemented in the submitted code.
- The local explanation method is occlusion-based; SHAP is listed as optional but is not the method used for the reported local explanations.

12. Deliverables / Project Files

- core.py - synthetic data generation, health score, status logic and XAI.
- train.py - training, model comparison, model selection and chart generation.
- app.py - Streamlit vehicle health dashboard.
- lstm_model.py - optional LSTM model.
- requirements.txt - project dependencies.
- outputs/vehicle_sensor_data.csv - generated synthetic telemetry.
- outputs/model_comparison.csv - recorded evaluation metrics.
- outputs/roc_curves.png, confusion_matrix.png, feature_importance.png - evaluation/XAI charts.
- outputs/sample_output.json - recorded example prediction.
- models/best_model.joblib - saved selected model and scaler.

13. Conclusion

The project demonstrates an end-to-end predictive-maintenance workflow from vehicle sensor data to failure probability, health scoring, maintenance alerts and explainability. On the supplied synthetic-data run, Logistic Regression achieved the highest ROC-AUC among the models actually evaluated and was saved as the deployed model. The implementation can be extended by replacing synthetic data with real telemetry and by evaluating the optional deep-learning and XGBoost paths in an environment where their dependencies are installed.